Additional Reading: Random Numbers and the Seed

Somewhere in this task you will see a line like `np.random.seed(42)` or `random.Random(42)`, and functions that take a `seed=` argument. The number 42 is a **seed**. This note explains what a seed is, why it is there, and two common mistakes to avoid.

Where random numbers come from

When an algorithm needs a random choice, such as a random restart or a random puzzle to test on, it asks the computer for a random number. Truly unpredictable numbers have to be measured from a physical process. Dedicated hardware does this by sampling something like thermal noise in a circuit, radioactive decay, or atmospheric noise, which is the source behind random.org.

The random generators we use in NumPy and Python's `random` work differently. They run a fast, fixed recipe that turns one number into the next, over and over, producing a long sequence with no obvious pattern:

$$\text{start} \longrightarrow 0.37 \longrightarrow 0.95 \longrightarrow 0.73 \longrightarrow 0.60 \longrightarrow \cdots$$

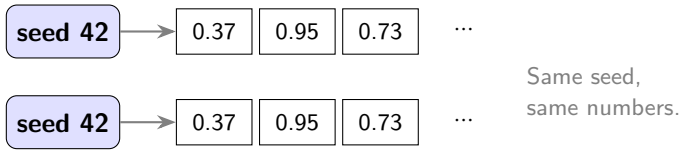
The sequence *looks* random, and that is enough for our algorithms, but it is completely determined by where the recipe started. Numbers made this way are called **pseudorandom**: 'pseudo' means 'looks like, but is not quite.' They look random; underneath they are a fixed recipe.

You cannot prove the numbers are not truly random just by looking at them. A good pseudorandom sequence passes the usual statistical tests for randomness: nothing in the numbers themselves gives the recipe away. We call them 'not random' because we *know* a fixed recipe and starting point produced them, not because they look any different from the real thing.

The seed is where the recipe starts

The **seed** is the number the recipe starts from. It fixes the whole sequence that follows.

- **Same seed → same sequence, every time.** Start the recipe at 42 today and again tomorrow, on your laptop or on the marker's, and you get the *identical* stream of 'random' numbers.
- **Different seed → a different sequence.** Start at 7 instead of 42 and you get a different stream, equally random-looking.



Why we *want* that

A sequence you can reproduce sounds like the opposite of random, but it is what you want when you are building and grading code:

- **Debugging.** If your program misbehaves, a fixed seed makes it misbehave the *same way* every run, so you can find the cause. A truly random bug that comes and goes is far harder to chase.
- **Grading and sharing.** Your marker can re-run your notebook and see the same numbers you reported. Your result is checkable.
- **Fair comparison.** When you compare two algorithms, giving them the *same* seed means they face the *same* random inputs. Any difference in the result is due to the algorithms, not to luck.

So we set a seed to make a random experiment **repeatable**.

Two mistakes to avoid

Mistake 1: set the seed once, not inside a loop that should vary

Setting the seed rewinds the recipe back to the start. If you set the same seed *inside* a loop, every pass rewinds to the same place and draws the *same* 'random' number, so a loop that was supposed to try something different each time does the exact same thing every time.

```
Wrong:  for each restart:
            set seed to 42          <- rewinds every time
            pick a random start      -> identical start on every restart!

Right:  set seed to 42              <- once, at the very top
        for each restart:
            pick a random start      -> a genuinely different start each time
```

Seed **once**, at the top of your experiment, then let the sequence run. Re-running the whole thing still gives the same results, and the values genuinely vary from one step to the next.

Mistake 2: two toolkits, two ways to seed

This unit uses two different random toolkits, and they are seeded differently. They do the same job; do not mix them up.

Toolkit	How to seed	What it is
NumPy	`np.random.seed(42)`	Sets one **shared** generator used by all of NumPy.
Python's `random`	`rng = random.Random(42)`	Makes your **own** generator; you call `rng.random()`, `rng.randint(...)`.

The `random.Random(42)` form gives you a private generator that you keep in a variable, often called `rng`. It does not disturb anyone else's randomness, which helps when different parts of a program each want their own repeatable stream. The `np.random.seed(42)` form sets a single generator that the whole NumPy library shares.

When a helper function takes a `seed=` argument, you are handing it a starting number to rewind the recipe, the same idea passed in as an argument. In this unit that argument acts on the **shared** NumPy generator above: the function runs `np.random.seed(your_number)` for you, so passing a seed rewinds NumPy's sequence for the rest of the program, well beyond that one function.

The `seed=None` default

You will often see the argument written as `seed=None`, and `None` is what you get when you call the function without a seed. `None` is Python's way of saying 'no value given.' Here it means: do not rewind the recipe, just keep drawing from wherever the shared sequence already is.

This default is what makes Mistake 1 work in your favour. You set the seed **once** at the top of your experiment, then call the function many times in a loop and leave the seed alone (`seed=None`). Each call continues the same running sequence, so every pass gets a genuinely different result, while re-running the whole notebook from the top still reproduces every number.

Pass a real number, say `seed=7`, only when you want to pin a **single** call, for example to reproduce one interesting case while you debug it. That rewinds the shared sequence back to that start, so use it deliberately, and never inside the loop that is meant to vary (Mistake 1 again).